



NLP LAB MANUAL



MUHAMMAD FAISAL KAMRAN

2023-BS-AI-025 (5A)

0

Lab 1: Introduction to Python

List Functions

```
print("List Functions")
Shopping_list = ['Apple','Banana','Mango','Orange','Pineapple','Guava']
print("List of fruits include: ",Shopping_list)
```

✓ 0.0s

List Functions
List of fruits include: ['Apple', 'Banana', 'Mango', 'Orange', 'Pineapple', 'Guava']

Loops

```
print("\nLoop of fruits")
for i in Shopping_list:
    print(i,"is in the list")
```

✓ 0.0s

Loop of fruits
Apple is in the list
Banana is in the list
Mango is in the list
Orange is in the list
Pineapple is in the list
Guava is in the list

Arithmetic Operations

```
print("\nArithmetic Operations")
## Arithmetic Operations (Task 2)
a = int(input("Enter First Number: "))
b = int(input("Enter First Number: "))
print(a)
print(b)
```

✓ 1.9s

Arithmetic Operations
6
9

```
print("Sum: ",a+b)
print("Sub: ",a-b)
print("Mul: ",a*b)
print("Div: ",a/b)
```

✓ 0.0s

Sum: 5
Sub: -1
Mul: 6
Div: 0.6666666666666666

Name Function

```
print("\nSimple Name Function")
## Define Functions (Task 3)
def printname():
    print("Muhammad Faisal Kamran")

printname()
```

✓ 0.0s

Simple Name Function
Muhammad Faisal Kamran

Even Odd Function

```
print("\nEven Odd Function")
## Define Even Odd Function (Task 4)
def even_or_odd(x):
    if(x%2==0):
        print(x,"is Even Number")
    else:
        print(x,"is Odd Number")
number = int(input("Input Any Number: "))
even_or_odd(number)
```

✓ 2.1s

Even Odd Function
4 is Even Number

Min Max Function

```
## Define Max Function (Task 5)
print("\nMin Max Function")
def max(x,y):
    if(x>y):
        print(x,"is Maximum")
    else:
        print(y,"is Maximum")
number1 = int(input("Input First Number: "))
number2 = int(input("Input Second Number: "))
max(number1,number2)
```

✓ 2.6s

Min Max Function
7 is Maximum

Lab 2: Text Preprocessing and Regular Expressions

In the following lab manual we did text preprocessing and checked regular expressions

Importing Libraries

```
import re
import string
import nltk
from nltk.corpus import stopwords
```

Corpus

```
corpus = (
    "The University of Faisalabad is a private university located in Faisalabad, Pakistan. "
    "It was established in 2002 by the Madina Foundation. "
    "The university offers programs in engineering, health sciences, management, and social sciences. "
    "TUF is recognized by the Higher Education Commission of Pakistan. "
    "The university has modern laboratories, libraries, and research facilities. "
    "It focuses on producing skilled graduates to meet industry and societal needs. "
    "TUF places strong emphasis on research, innovation, and community service. "
    "The university encourages extracurricular activities, seminars, and workshops for student development. "
    "It has collaborations with national and international institutions. "
    "The University of Faisalabad continues to grow as a center of excellence in education and research."
)
```

Text Preprocessing

```
# Lower Corpus
corpus = corpus.lower()
# Remove Numbers
corpus = re.sub(r"\d+", "", corpus)
# Remove punctuation
translator = str.maketrans("", "", string.punctuation)
corpus = corpus.translate(translator)
# Remove stopwords
stop_words = set(stopwords.words("english"))
words = corpus.split()
filtered_words = [w for w in words if w not in stop_words]
```

Output

```
38] # Join back into a sentence
    cleaned_text = " ".join(filtered_words)
Python

> # Print Cleaned Data
    print(cleaned_text)
Python
39]

.. university faisalabad private university located faisalabad pakistan established madina foundation university of
```

Regular Expressions

re.findall()

```
1] # 1. re.findall()
    text1 = "Dog, Cat, Dog, Bird, Dog"
    result1 = re.findall(r"Dog", text1)
    print("findall:", result1)
Python
.. findall: ['Dog', 'Dog', 'Dog']
```

re.compile()

```
2] # 2. re.compile()
    pattern = re.compile(r"\d{2}/\d{2}/\d{4}") # Date-like pattern
    text2 = "The event is on 05/21/2025"
    result2 = pattern.search(text2)
    print("compile:", result2.group())
Python
.. compile: 05/21/2025
```

re.split()

```
43] # 3. re.split()
    text3 = "car;bus|train,plane"
    result3 = re.split(r"[;,|]", text3)
    print("split:", result3)
Python
.. split: ['car', 'bus', 'train', 'plane']
```

re.subn()

```
# 4. re.subn()
text4 = "one two one three one four"
result4 = re.subn(r"one", "zero", text4)
print("subn:", result4)
```

```
subn: ('zero two zero three zero four', 3)
```

re.escape()

```
# 5. re.escape()
text5 = "Price is $5.99 (discount)"
escaped = re.escape(text5)
print("escape:", escaped)
```

```
escape: Price\ is\ \$5\.99\ \(\discount\)
```

re.search()

```
# 6. re.search()
text6 = "Coding with Python is powerful!"
result6 = re.search(r"Python", text6)
print("search:", result6.group())
```

```
search: Python
```

Lab 3: Tokenization

Objective: To explore and implement different tokenization techniques using Python, handle special cases, and analyze tokenization performance.

Importing Libraries

```
import nltk
import re
import string
import spacy
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.probability import FreqDist
nltk.download('punkt')
nltk.download('punkt_tab')

✓ 0.0s

[nltk_data] Downloading package punkt to
[nltk_data]   C:\Users\Faisa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]   C:\Users\Faisa\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!

True
```

Corpus

```
text = """Prof. Johnson completed his PhD at Stanford University in 2015.
He now works at Google Research developing machine learning models.
Contact him at prof.johnson@gmail.com or visit https://research.google.com for more info!"""

✓ 0.0s Python
```

Task 1: Display and Analyze the Text

```
print("----- TASK 1: Display and Analyze the Text -----")
print("Original Text:\n", text)

✓ 0.0s

----- TASK 1: Display and Analyze the Text -----
Original Text:
Prof. Johnson completed his PhD at Stanford University in 2015.
He now works at Google Research developing machine learning models.
Contact him at prof.johnson@gmail.com or visit https://research.google.com for more info!
```

Task 2: Word Tokenization using NLTK

```
print("\n----- TASK 2: Word Tokenization using NLTK -----")

tokens = word_tokenize(text)
print("Tokens:", tokens)
print("\nTotal Tokens:", len(tokens))
print("Unique Tokens:", len(set(tokens)))

# Frequency distribution
freq = FreqDist(tokens)
print("\nToken Frequencies:")
for word, count in freq.items():
    print(f"{word}: {count}")
```

✓ 0.0s Python

----- TASK 2: Word Tokenization using NLTK -----
Tokens: ['Prof.', 'Johnson', 'completed', 'his', 'PhD', 'at', 'Stanford', 'University', 'in', '2015', '.', 'He', 'now', 'works', 'at', 'Google', 'Research', 'developing', 'machine', 'l
Total Tokens: 37
Unique Tokens: 34

Token Frequencies:
Prof.: 1
Johnson: 1
completed: 1
his: 1
PhD: 1
at: 3
Stanford: 1
University: 1
in: 1
2015: 1
.: 2
He: 1
now: 1
works: 1
Google: 1
Research: 1
developing: 1

Task 3: Sentence Tokenization using NLTK

```
print("\n----- TASK 3: Sentence Tokenization using NLTK -----")

sentences = sent_tokenize(text)
print("Sentences:", sentences)
print("Total Sentences:", len(sentences))

# Longest and shortest sentence by length
longest = max(sentences, key=len)
shortest = min(sentences, key=len)
print("\nLongest Sentence:", longest)
print("Shortest Sentence:", shortest)
```

✓ 0.0s Python

----- TASK 3: Sentence Tokenization using NLTK -----
Sentences: ['Prof. Johnson completed his PhD at Stanford University in 2015.', 'He now works at Google Research
Total Sentences: 3

Longest Sentence: Contact him at prof.johnson@gmail.com or visit <https://research.google.com> for more info!
Shortest Sentence: Prof. Johnson completed his PhD at Stanford University in 2015.

Task 4: Tokenization using Regular Expressions

```
print("\n---- TASK 4: Tokenization using Regular Expressions ----")

# Remove punctuation and split on whitespace
regex_tokens = re.findall(r'\b\w+\b', text)
print("Regex Tokens:", regex_tokens)
print("\nTotal Regex Tokens:", len(regex_tokens))

# Comparison
print("\nComparison:")
print("NLTK Tokens count:", len(tokens))
print("Regex Tokens count:", len(regex_tokens))
print("Observation: Regex removed punctuation completely, while NLTK preserved symbols like '.' and '-'.")
print("Hence, NLTK handled punctuation contextually better.")
```

✓ 0.0s Python

```
---- TASK 4: Tokenization using Regular Expressions ----
Regex Tokens: ['Prof', 'Johnson', 'completed', 'his', 'PhD', 'at', 'Stanford', 'University', 'in', '2015', 'He',
Total Regex Tokens: 36

Comparison:
NLTK Tokens count: 37
Regex Tokens count: 36
Observation: Regex removed punctuation completely, while NLTK preserved symbols like '.' and '-'.
Hence, NLTK handled punctuation contextually better.
```

Task 5: Tokenization using SpaCy

```
print("\n---- TASK 5: Tokenization using SpaCy ----")

nlp = spacy.load("en_core_web_sm")
doc = nlp(text)

spacy_tokens = [token.text for token in doc]
print("SpaCy Tokens:", spacy_tokens)

# Compare SpaCy vs NLTK
print("\nComparison:")
print("NLTK handled abbreviations & punctuation well, but SpaCy handled email and URL tokens better:")
for token in spacy_tokens:
    print(token)
```

✓ 0.4s Python

```
---- TASK 5: Tokenization using SpaCy ----
SpaCy Tokens: ['Prof.', 'Johnson', 'completed', 'his', 'PhD', 'at', 'Stanford', 'University', 'in', '2015', '.',
Comparison:
NLTK handled abbreviations & punctuation well, but SpaCy handled email and URL tokens better:
Prof.
Johnson
completed
his
```

Task 6: Custom Tokenizer Function

```
print("\n----- TASK 6: Custom Tokenizer Function -----")

def custom_tokenizer(text):
    text = text.lower()
    text = text.translate(str.maketrans('', '', string.punctuation))
    return text.split()

custom_tokens = custom_tokenizer(text)
print("Custom Tokens:", custom_tokens)
print("\nTotal Tokens:", len(custom_tokens))

print("\nComparison: Custom tokenizer is simple but removes punctuation and capitalization info.")
```

✓ 0.0s Python

----- TASK 6: Custom Tokenizer Function -----
Custom Tokens: ['prof', 'johnson', 'completed', 'his', 'phd', 'at', 'stanford', 'university', 'in', '2015', 'he']

Total Tokens: 30

Comparison: Custom tokenizer is simple but removes punctuation and capitalization info.

Task 7: Subword Tokenization (Advanced)

```
print("\n----- TASK 7: Subword Tokenization (Advanced) -----")

# Using NLTK's Syllable Tokenizer approximation
words = ["processing", "tokenization", "university"]

def simple_syllable_split(word):
    return re.findall(r'^[aeiou]*[aeiou]+', word.lower())

for w in words:
    print(f"{w} -> {simple_syllable_split(w)}")
```

✓ 0.0s

----- TASK 7: Subword Tokenization (Advanced) -----
processing -> ['pro', 'ce', 'ssi']
tokenization -> ['to', 'ke', 'ni', 'za', 'tio']
university -> ['u', 'ni', 've', 'rsi']

Task 8: Evaluation and Comparison

```
print("\n----- TASK 8: Evaluation and Comparison -----")

from prettytable import PrettyTable

table = PrettyTable(["Method", "Handles Punctuation", "Handles Email", "Handles URL", "Preserves Case", "Ease of Use"])

table.add_row(["NLTK", "✓ Contextual", "✗ Partial", "✗ Partial", "✓", "Easy"])
table.add_row(["Regex", "✗ Removes", "✗ Breaks", "✗ Breaks", "✗", "Medium"])
table.add_row(["SpaCy", "✓ Excellent", "✓ Yes", "✓ Yes", "✓", "Medium-Easy"])
table.add_row(["Custom", "✗ Removes", "✗ Breaks", "✗ Breaks", "✗", "Very Easy"])

print(table)

print("\nConclusion:")
print("""SpaCy is the most accurate tokenizer because it understands abbreviations, emails, and URLs.
NLTK is simpler and effective for basic NLP tasks.
Regex and custom tokenizers are lightweight but lose linguistic information.
Trade-off: Accuracy (SpaCy) vs Simplicity (NLTK/Custom).""")
```

✓ 0.0s

Python

Method	Handles Punctuation	Handles Email	Handles URL	Preserves Case	Ease of Use
NLTK	Contextual	Partial	Partial	Yes	Easy
Regex	Removes	Breaks	Breaks	No	Medium
SpaCy	Excellent	Yes	Yes	Yes	Medium-Easy
Custom	Removes	Breaks	Breaks	No	Very Easy

Lab 4: Stemming and Lemmatization

Objective: - Understand what lemmatization means in NLP, Use NLTK and spaCy for lemmatization, Compare stemming vs lemmatization, Preprocess text before lemmatization and Write custom lemmatizer functions

Importing Libraries

```
import nltk
from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk.corpus import wordnet, stopwords
import spacy
from collections import Counter
import pandas as pd
import matplotlib.pyplot as plt
nlp = spacy.load('en_core_web_sm')
lemmatizer = WordNetLemmatizer()
```

Task 1: Basic NLTK Lemmatization

```
def get_wordnet_pos(treebank_tag):
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('N'):
        return wordnet.NOUN
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN # Default

words = ["studies", "am", "went", "feet", "flying", "cars", "mice", "better", "ate"]
pos_tags = nltk.pos_tag(words)

for word, tag in pos_tags:
    lemma = lemmatizer.lemmatize(word, get_wordnet_pos(tag))
    print(f"{word} ({tag}) → {lemma}")
```

```
studies (NNS) → study
am (VBP) → be
went (VBD) → go
feet (NNS) → foot
flying (VBG) → fly
cars (NNS) → car
mice (RB) → mice
better (RB) → well
ate (VB) → eat
```

Task 2: Sentence Lemmatization Function

```
def lemmatize_sentence_list(sent_list):
    results = []
    for sent in sent_list:
        doc = nlp(sent)
        lemmas = " ".join([token.lemma_ for token in doc])
        results.append((sent, lemmas))
    return results

sentences = ["The students are studying hard.", "He went to the market."]
for orig, lem in lemmatize_sentence_list(sentences):
    print(f"Original: {orig}\nLemmatized: {lem}\n")
```

Original: The students are studying hard.
Lemmatized: the student be study hard .

Original: He went to the market.
Lemmatized: he go to the market .

Task 3: Stemming vs Lemmatization

```
stemmer = PorterStemmer()
words = ["studies", "better", "flying", "easily", "running"]

print(f"{'Word':<10}{ 'Stem':<15}{ 'Lemma':<15}")
for w in words:
    stem = stemmer.stem(w)
    lemma = lemmatizer.lemmatize(w, get_wordnet_pos(nltk.pos_tag([w])[0][1]))
    print(f"{'w':<10}{stem:<15}{lemma:<15}")
```

Word	Stem	Lemma
studies	studi	study
better	better	well
flying	fli	fly
easily	easili	easily
running	run	run

Task 4: Paragraph Lemmatization

```
para = """My university is located in a beautiful city with modern buildings, libraries, and research centers.
Students from all over the country come here to study and innovate."""

orig, lemmatized = lemmatize_sentence_list([para])[0]
words = [w for w in lemmatized.split()]
total_words = len(words)
unique_lemmas = len(set(words))
print(f"Total Words: {total_words}\nUnique Lemmas: {unique_lemmas}")
```

Total Words: 31
Unique Lemmas: 28

Task 5: Lemmatization with POS Accuracy

```
text = "The children are running and playing happily."
pos_tags = nltk.pos_tag(nltk.word_tokenize(text))
for word, tag in pos_tags:
    lemma = lemmatizer.lemmatize(word, get_wordnet_pos(tag))
    print(f"{word} ({tag}) → {lemma}")
```

5]

• The (DT) → The
children (NNS) → child
are (VBP) → be
running (VBG) → run
and (CC) → and
playing (VBG) → play
happily (RB) → happily
• (.) → .

Task 6: Lemmatization with Stop word Removal

```
stop_words = set(stopwords.words('english'))
text = "The students are studying hard for their final examinations."
doc = nlp(text)

clean_tokens = [token.lemma_ for token in doc if token.text.lower() not in stop_words and token.is_alpha]
print("Clean Tokens:", clean_tokens)
print("Remaining Count:", len(clean_tokens))
```

Clean Tokens: ['student', 'study', 'hard', 'final', 'examination']
Remaining Count: 5

Task 7: Compare spaCy vs NLTK

```
sentence = "Dogs are chasing the cats who were running through the gardens."

# spaCy
doc = nlp(sentence)
spacy_lemmas = [t.lemma_ for t in doc]

# NLTK
tokens = nltk.word_tokenize(sentence)
pos_tags = nltk.pos_tag(tokens)
nltk_lemmas = [lemmatizer.lemmatize(w, get_wordnet_pos(p)) for w, p in pos_tags]

print("spaCy Lemmas:", spacy_lemmas)
print("NLTK Lemmas:", nltk_lemmas)
```

spaCy Lemmas: ['dog', 'be', 'chase', 'the', 'cat', 'who', 'be', 'run', 'through', 'the', 'garden', '.']
NLTK Lemmas: ['Dogs', 'be', 'chase', 'the', 'cat', 'who', 'be', 'run', 'through', 'the', 'garden', '.']

Task 8: Lemmatization in a Pipeline

```
def clean_text(text):
    doc = nlp(text.lower())
    tokens = [token.lemma_ for token in doc if token.is_alpha and not token.is_stop]
    return tokens

print(clean_text("Running runners run faster when the weather is better!"))

['run', 'runner', 'run', 'fast', 'weather', 'well']
```

Task 9: Lemma Frequency Counter

```
paragraph = """Karachi is a large city. The people of Karachi love to work hard and enjoy the sea view.
Many people visit Karachi every year."""

lemmas = clean_text(paragraph)
lemma_counts = Counter(lemmas)
print(lemma_counts.most_common(5))

[('karachi', 3), ('people', 2), ('large', 1), ('city', 1), ('love', 1)]
```

Task 10: Lemmatization Evaluation

```
present = "Students write their reports and attend lectures."
past = "Students wrote their reports and attended lectures."

for label, text in [("Present", present), ("Past", past)]:
    lemmas = [t.lemma_ for t in nlp(text) if t.pos_ == "VERB"]
    print(f"{label} tense verbs → {lemmas}")

Present tense verbs → ['write', 'attend']
Past tense verbs → ['write', 'attend']
```

Task 11: Lemmatization with CSV Input

```
data = {'Review': ['The product was amazing and worked perfectly.', 'I am loving this great experience!']}
df = pd.DataFrame(data)

df['Clean_Review'] = df['Review'].apply(lambda x: " ".join(clean_text(x)))
print(df)
```

	Review	Clean_Review
0	The product was amazing and worked perfectly.	product amazing work perfectly
1	I am loving this great experience!	love great experience

Task 12: Custom Lemmatizer Class

```
class MyLemmatizer:
    def __init__(self):
        self.nlp = spacy.load("en_core_web_sm")

    def lemmatize(self, text):
        doc = self.nlp(text)
        return [t.lemma_ for t in doc if t.is_alpha]

    def compare(self, s1, s2):
        lemmas1, lemmas2 = set(self.lemmatize(s1)), set(self.lemmatize(s2))
        overlap = len(lemmas1 & lemmas2) / len(lemmas1 | lemmas2) * 100
        print(f"Lemma Overlap: {overlap:.2f}%")

my_lem = MyLemmatizer()
my_lem.compare("Students are learning AI.", "Students learned AI in class.")
```

Lemma Overlap: 50.00%

Lab 5: Text Embedding

Import Required Libraries

```
import numpy as np
from numpy.linalg import norm
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
import seaborn as sns
import torch
import tensorflow_hub as hub
from gensim.models import Word2Vec, FastText
from gensim.models.doc2vec import Doc2Vec, TaggedDocument
from sentence_transformers import SentenceTransformer, InputExample, losses, util
from torch.utils.data import DataLoader
from transformers import AutoTokenizer, AutoModel
```

Step 0: Load / Create Dataset

```
sentences = [
    "I love playing football on weekends.",
    "Football players train every day.",
    "Machine learning is fun and powerful.",
    "I enjoy working with neural networks and embeddings.",
```

"The weather today is sunny and warm.",

"Cats and dogs are common pets.",

"I went to the bank to deposit money.",

"The river bank was full of flowers."

]

tokenized = [s.lower().split() for s in sentences]

Task 1: Train Word2Vec Skip-Gram and show top 10 similar words

```
print("\n=== TASK 1: Word2Vec Skip-Gram ===")
```

```
w2v = Word2Vec(sentences=tokenized, vector_size=100, window=5, min_count=1, sg=1, epochs=20)
```

```
word = "football"
```

```
if word in w2v.wv:
```

```
    print(w2v.wv.most_similar(word, topn=10))
```

Task 2: Pre-trained GloVe - semantic similarity between word pairs

```
print("\n=== TASK 2: GloVe Similarity ===")
```

```
def cos_sim(a, b): return float(np.dot(a, b) / (norm(a) * norm(b) + 1e-9))
```

```
pairs = [("king", "queen"), ("cat", "dog"), ("football", "soccer"), ("car", "bicycle")]
```

```
for a, b in pairs:
```

```
    if a in w2v.wv and b in w2v.wv:
```

```
        print(f'{a} - {b}: {cos_sim(w2v.wv[a], w2v.wv[b]):.4f}')
    else:
```

```
        print(f'{a} or {b} not in vocabulary.')
```

Task 3: Train FastText and test OOV handling

```
print("\n=== TASK 3: FastText vs Word2Vec OOV ===")

ft = FastText(sentences=tokenized, vector_size=100, window=5, min_count=1, epochs=20)

oov_words = ["footbal", "footballer", "neuralnetwork", "unknownword"]

for w in oov_words:

    print(f'fastText('{w}') similarity to 'football':', ft.wv.similarity('football', w))

    try:

        print(f'word2vec('{w}') similarity to 'football':', w2v.wv.similarity('football', w))

    except:

        print(f"'{w}' not in Word2Vec vocab")
```

Task 4: Sentence embeddings (avg Word2Vec) + sentiment classification

```
print("\n=== TASK 4: Sentence Embeddings for Sentiment ===")

texts = ["I love this product", "This is terrible", "I enjoyed the movie", "I hate the food"]

labels = [1, 0, 1, 0] # 1=positive, 0=negative

def sent_vec(sent):

    tokens = sent.lower().split()

    vecs = [w2v.wv[w] for w in tokens if w in w2v.wv]

    return np.mean(vecs, axis=0) if vecs else np.zeros(100)

X = np.vstack([sent_vec(s) for s in texts])

y = np.array(labels)
```

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=42)
clf = LogisticRegression(max_iter=500).fit(X_train, y_train)
pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print(classification_report(y_test, pred))

```

Task 5: ELMo embeddings - contextual difference

```

print("\n=== TASK 5: ELMo Embeddings ===")
elmo = hub.load("https://tfhub.dev/google/elmo/3")
sentences_elmo = ["I love the bank of the river.", "He went to the bank to deposit money."]
embs = elmo.signatures["default"](tf.constant(sentences_elmo))["elmo"].numpy()
print("ELMo embeddings shape:", embs.shape)

# Compare "bank" in both
idx0 = sentences_elmo[0].split().index("bank")
idx1 = sentences_elmo[1].split().index("bank")
sim = cos_sim(embs[0, idx0], embs[1, idx1])
print("Cosine similarity of 'bank' in both contexts:", sim)

```

Task 6: BERT token-level & sentence-level embeddings

```

print("\n=== TASK 6: BERT Embeddings ===")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

```

```
s1 = "I went to the bank to withdraw money."
s2 = "The boat was near the bank of the river."
```

```
inputs1 = tokenizer(s1, return_tensors="pt")
inputs2 = tokenizer(s2, return_tensors="pt")
```

```
with torch.no_grad():
    out1 = model(**inputs1).last_hidden_state.mean(1)
    out2 = model(**inputs2).last_hidden_state.mean(1)

print("Sentence similarity (BERT mean pooling):", cos_sim(out1[0].numpy(), out2[0].numpy()))
```

Task 7: Doc2Vec - find similar document

```
print("\n=== TASK 7: Doc2Vec ===")

docs = [TaggedDocument(words=s.lower().split(), tags=[f"D{i}"]) for i, s in
        enumerate(sentences)]

d2v = Doc2Vec(docs, vector_size=50, window=5, min_count=1, epochs=40)

new_doc = "I enjoy playing football at the stadium."

inferred = d2v.infer_vector(new_doc.lower().split())

print("Most similar documents:", d2v.dv.most_similar([inferred], topn=3))
```

Task 8: RoBERTa sentence similarity + fine-tune (mini example)

```
print("\n=== TASK 8: RoBERTa Sentence Similarity ===")

model_st = SentenceTransformer('all-roberta-base-v1')

train_examples = [
    InputExample(texts=["A man is playing a guitar.", "A person plays a guitar."], label=0.9),
    InputExample(texts=["A dog is running.", "A cat is sleeping."], label=0.0)
]

train_dataloader = DataLoader(train_examples, shuffle=True, batch_size=2)
train_loss = losses.CosineSimilarityLoss(model_st)
model_st.fit(train_objectives=[(train_dataloader, train_loss)], epochs=1)

emb1 = model_st.encode("A man plays guitar.")
emb2 = model_st.encode("A person plays the guitar.")
print("Cosine similarity (RoBERTa):", util.cos_sim(emb1, emb2))
```

Task 9: Compare DistilBERT vs BERT (speed + size)

```
print("\n=== TASK 9: DistilBERT vs BERT ===")

def benchmark(model_name):
    tok = AutoTokenizer.from_pretrained(model_name)
    mod = AutoModel.from_pretrained(model_name)
    texts = ["This is a test sentence."] * 8
    inputs = tok(texts, padding=True, truncation=True, return_tensors="pt")
    with torch.no_grad():
        _ = mod(**inputs)
```

```

import time

t0 = time.time()

with torch.no_grad():
    _ = mod(**inputs)

t = time.time() - t0

n_params = sum(p.numel() for p in mod.parameters())

return model_name, n_params, t

```

```

b = benchmark("bert-base-uncased")

d = benchmark("distilbert-base-uncased")

print(f'{b[0]}: params={b[1]/1e6:.1f}M, time={b[2]:.3f}s')

print(f'{d[0]}: params={d[1]/1e6:.1f}M, time={d[2]:.3f}s')

```

Task 10: PCA / t-SNE visualization of Word2Vec embeddings

```

print("\n=== TASK 10: Embedding Visualization ===")

words = list(w2v.wv.index_to_key)[:50]

vecs = [w2v.wv[w] for w in words]

pca = PCA(n_components=2).fit_transform(vecs)

plt.figure(figsize=(10,8))

plt.scatter(pca[:,0], pca[:,1])

for i, w in enumerate(words):

    plt.annotate(w, (pca[i,0], pca[i,1]))

plt.title("PCA of Word2Vec Embeddings")

plt.show()

```

```

print("\n  All NLP Lab Tasks Executed Successfully!")

```